

Applied Machine Learning

Unit 04 — Lecture 07 Notes

Data Splitting, Cross-Validation and Imbalanced Data

Tofik Ali

Autumn 2026

Read this first

These notes are written so that you can learn the material *without* having been in the lecture. Every concept is developed from scratch, every number you see was produced by running the code shown beside it, and every figure was generated from real computation rather than drawn by hand.

The previous lecture chose the columns: scaling, encoding, principal components, feature selection. It ended on one measurement — a logistic regression scored 0.8475 on 500 columns of pure noise with coin-flip labels, because the features had been selected before the data was split. That was a symptom. This lecture is about the disease: **the split itself** — what goes in it, when it happens, and what it is allowed to know.

Four questions organise the lecture. How reliable is a **single** held-out score? What do I do when the rows are **not independent**? How many **folds**, and how do I tune without cheating? And what do I do when one class is **one per cent** of the data?

The single result to carry out of this lecture: the same three lines of code reported $F_1 = 0.8942$ or $F_1 = 0.1133$ depending on whether one of them sat inside or outside the cross-validation loop. Work through Section 6.6 with a Python session open until you can say exactly why.

Contents

1	How to use these notes	2
1.1	What you need	2
1.2	Learning outcomes	4
2	A single train/test split	4
2.1	What a held-out score is actually estimating	4
2.2	By hand: twelve rows, four held out	4
2.3	The split is a random variable: five hundred seeds	5
2.4	It is not just noisy — it ranks models wrongly	7
3	Split strategies: stratified, grouped, ordered	7
3.1	Stratification, by hand and in code	8
3.2	A fold with nothing to learn from	8
3.3	Grouped rows: the unit of independence	10
3.4	Time-ordered rows	11
4	Cross-validation	12

4.1	The recipe, and its cost	12
4.2	Choosing k : bias, variance and time	13
4.3	Variance of the estimate, measured properly	14
4.4	More folds, or more repeats?	15
5	Tuning honestly: validation sets and nested CV	16
5.1	The moment you tune, the score stops being an estimate	16
5.2	Predict the number first: tuning on pure noise	16
5.3	The same effect where there is real signal	17
5.4	Nested cross-validation	18
6	Imbalanced data	18
6.1	By hand: one thousand patients, ten of them ill	18
6.2	The same thing measured	19
6.3	Four things you can do about it	20
6.4	SMOTE is one line of arithmetic	21
6.5	Does any of it help?	22
6.6	The mistake: resampling before the split	23
6.7	The whole thing in one place	24
7	Summary	25
8	Practice questions	26
9	Solutions	27
10	References and further reading	29

1 How to use these notes

Read in order. Section 4 assumes the sampling-noise argument of Section 2; Section 5 assumes Section 4; and Section 6 uses the stratification material of Section 3 throughout. Everything here assumes only what Lecture 6 assumed: a fitted model, a score, and a `Pipeline`.

There are four kinds of box. **Key idea** — the one sentence to remember a month from now. **Worked example** — a calculation done by hand, with small numbers, before any library is used. **Caution** — the specific mistake that section exists to prevent. **Try it yourself** — something to run, with the expected output given so you can check.

Code appears in a framed block, and directly beneath it a shaded block showing what that code *actually printed*. If you run it and get something different, trust your machine and tell me. One exception is flagged where it occurs: wall-clock timings depend on your hardware and vary between runs even on the same machine.

1.1 What you need

```
pip install numpy scipy scikit-learn matplotlib
```

Every number in these notes was produced with:

```
import numpy, scipy, sklearn, matplotlib
print("numpy", numpy.__version__, "| scipy", scipy.__version__,
      "| sklearn", sklearn.__version__,
      "| matplotlib", matplotlib.__version__)
```

imbalanced-learn is not required. Section 6.4 implements SMOTE in fifteen lines, both so that the lecture does not depend on a package your lab machine may not have, and because writing it out is the fastest way to understand what it does. If **imbalanced-learn** is installed on your machine you are welcome to use it; the results will be the same up to the random draw.

Four bundled datasets recur, and **none of them downloads anything**.

```
from sklearn.datasets import (load_breast_cancer, load_wine,
                              load_digits, load_iris)
bc = load_breast_cancer(); Xb, yb = bc.data, bc.target
wine = load_wine(); Xw, yw = wine.data, wine.target
dg = load_digits(); Xg, yg = dg.data, dg.target
iris = load_iris(); Xi, yi = iris.data, iris.target
print("class balance, breast_cancer:", np.bincount(yb).tolist())
```

```
load_breast_cancer  569 rows x 30 features, 2 classes
load_wine           178 rows x 13 features, 3 classes
load_digits         1797 rows x 64 features, 10 classes
load_iris           150 rows x 4 features, 3 classes
class balance, breast_cancer: [212, 357]
class balance, wine           : [59, 71, 48]
```

The imbalanced experiments use a fixed-seed synthetic problem built with **make_classification**, because no bundled dataset is skewed enough to make the point: 5000 rows, 20 features, and a minority class we can dial from 50% down to 0.2%.

Four datasets, one seed — the convention for this lecture

Everything in these notes is computed from the four bundled tables above and **one seed**. The seed is **0**: every split, every shuffle, every stochastic estimator and every resampler draw is created as `np.random.default_rng(0)` or given `random_state=0`, and every listing that involves chance shows that line rather than hiding it.

Two kinds of exception are deliberate, and this lecture is the one where they matter most. *Three synthetic datasets are defined by a seed rather than stirred by one*, so those seeds keep their own names and values: the 5000-row 99:1 problem, the 40-patient grouped study and the 600-day series. Change one of those and you change the data the section is about, not the experiment.

Most of this lecture is sweeps. Five hundred train/test splits, forty different shuffles of a k -fold, twenty nested trials: these deliberately use a different seed every repetition, because **the spread across repetitions is the entire subject of the lecture**. Collapsing them onto one seed would delete the result. Their seeds are derived from the same 0 — `range(0, 500)`, `range(0, 40)` — so the sweep as a whole still reproduces exactly.

So every number printed in these notes reproduces exactly if you run the code as written. The single exception is wall-clock time, and it is labelled where it appears.

Do not reach for fetch_*

Every **fetch_*** loader downloads over HTTPS, and on the campus network those downloads fail with a certificate error you cannot fix from your machine. Nothing in this course depends on one: use the bundled loaders (`load_breast_cancer`, `load_wine`, `load_digits`, `load_iris`, `load_diabetes`) or a fixed-seed synthetic set.

1.2 Learning outcomes

By the end of this lecture you should be able to:

1. Explain what a held-out score estimates, and demonstrate that a single train/test score is a random variable with a measurable spread (CO2).
2. Choose the right splitting strategy — plain, stratified, grouped or time-ordered — from the structure of the rows, and justify the choice (CO2).
3. Apply k -fold, stratified k -fold, repeated k -fold, shuffle-split and leave-one-out cross-validation, and reason about the bias, the variance and the cost of each (CO2).
4. Separate model *selection* from performance *estimation*, using either a three-way split or nested cross-validation, and quantify the optimism of getting this wrong (CO2).
5. Diagnose class imbalance, choose metrics that survive it, and apply random oversampling, random undersampling, SMOTE and class weights — inside the cross-validation loop (CO2).
6. Recognise and repair the three leakage patterns of this lecture: grouped rows split at random, ordered rows shuffled, and resampling performed before the split (CO2).

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.; Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.; James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.; Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.

2 A single train/test split

2.1 What a held-out score is actually estimating

You hold rows out because a model can memorise. Training accuracy measures memory; held-out accuracy measures *generalisation*. The quantity you want is the performance the model would have on **new rows drawn from the same source as your data**, and the held-out rows are a stand-in for those new rows.

The stand-in is chosen at random. That single sentence has a consequence most reports ignore: **the number you print is a random variable**. It has a mean and it has a spread, and the spread is usually larger than the difference you are trying to detect.

Key idea

`train_test_split(X, y, test_size=0.25, random_state=42)` — the last argument is not a formality. It is a coin toss with your result on it.

2.2 By hand: twelve rows, four held out

Before any code, do the arithmetic on a table small enough to see. Twelve patients, nine healthy (0) and three ill (1), and the file arrives sorted by label, which is how a database export usually arrives:

row	0	1	2	3	4	5	6	7	8	9	10	11
y	0	0	0	0	0	0	0	0	0	1	1	1

How many ill patients land in the test set?

Hold out four of the twelve rows.

Take the last four. You get rows 8,9,10,11: three ill patients out of three. The model trains on *no* ill patients at all, and the test set is 75% ill when the population is 25% ill. Both halves are wrong.

Shuffle first. The number of ill patients in the test set is hypergeometric: it can be 0, 1, 2 or 3. The probability of getting none is

$$P(0) = \frac{\binom{9}{4}}{\binom{12}{4}} = \frac{126}{495} = 0.2545,$$

so roughly one experiment in four trains on all three ill patients and has *nothing* to compute a recall from.

Stratify. Deal the two classes out separately, each in the ratio $4/12 = 1/3$. Ill patients: $3 \times \frac{1}{3} = 1$. Healthy patients: $9 \times \frac{1}{3} = 3$. Every test set has exactly one ill patient and three healthy ones, every time.

Now the same thing in code. Six seeds, with and without stratification:

```
y12 = np.array([0]*9 + [1]*3)
for lab, strat in (("stratify=None", None), ("stratify=y", y12)):
    out = [int(y12[train_test_split(np.arange(12), test_size=4,
                                   random_state=s, stratify=strat)[1]].sum())
            for s in range(6)]
    print(lab, "minority rows in the test set over 6 seeds:", out)
```

```
train_test_split on the same twelve rows, four rows held out
stratify=None    minority rows in the test set over 6 seeds: [2, 1, 1, 0, 1, 1]
stratify=y       minority rows in the test set over 6 seeds: [1, 1, 1, 1, 1, 1]
```

The hand calculation said “about one experiment in four gets zero”. Six honest experiments produced exactly one such case, seed 3. On that seed the recall is 0/0 and scikit-learn returns 0.0 with a warning that most people do not read.

2.3 The split is a random variable: five hundred seeds

Twelve rows is a toy. Do it on real data. `load_wine` has 178 rows, 13 features and 3 classes; fit a decision tree on a 75/25 split and change nothing but `random_state`.

```
for seed in range(12):
    Xtr, Xte, ytr, yte = train_test_split(Xw, yw, test_size=0.25,
                                          random_state=seed)
    m = DecisionTreeClassifier(random_state=0).fit(Xtr, ytr)
    print(" random_state=%2d test accuracy %.4f" % (seed, m.score(Xte, yte)))
```

```
random_state= 0    test accuracy 0.9333
random_state= 1    test accuracy 0.9556
random_state= 2    test accuracy 0.9556
random_state= 3    test accuracy 0.8667
random_state= 4    test accuracy 0.9111
random_state= 5    test accuracy 0.9111
random_state= 6    test accuracy 0.9333
random_state= 7    test accuracy 0.9333
random_state= 8    test accuracy 0.9333
random_state= 9    test accuracy 0.9556
random_state=10    test accuracy 0.9111
random_state=11    test accuracy 0.9556
```

Nine points of accuracy between the best seed and the worst, and these are only the first twelve. Run five hundred:

```
def split_spread(X, y, model, n=500, test_size=0.25):
    a = np.empty(n)
    for s in range(n):
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=test_size,
                                              random_state=s)
        a[s] = clone(model).fit(Xtr, ytr).score(Xte, yte)
    return a

acc = split_spread(Xw, yw, DecisionTreeClassifier(random_state=0))
print("mean %.4f sd %.4f min %.4f max %.4f" %
      (acc.mean(), acc.std(), acc.min(), acc.max()))
print("2.5th-97.5th percentile", np.percentile(acc, [2.5, 97.5]))
```

```
wine, decision tree, 500 different random_state values
mean 0.9132 sd 0.0425 min 0.7556 max 1.0000 range 0.2444
2.5th-97.5th percentile [0.8106, 0.9778]
the test set is 44 rows, so one row is worth 0.0227 of accuracy
5-fold cross-validation instead: 0.9273 +/- 0.0513
```

Key idea

A paper reporting **1.0000** and a paper reporting **0.7556** could have run *exactly the same code*. The middle 95% of single-split results on this dataset spans 17 points of accuracy. “We achieved 95.6% accuracy” is not a result; it is one draw from a distribution.

Why so wide? Because the test set has 44 rows, so one patient is worth $1/44 = 0.0227$ of accuracy. Three patients changing sides moves the headline by seven points. The standard error of an accuracy estimated on n_{te} independent rows is

$$SE = \sqrt{\frac{p(1-p)}{n_{te}}},$$

which for $p = 0.91$ and $n_{te} = 44$ gives 0.043 — almost exactly the measured 0.0425. The measurement and the formula agree, which is worth noticing: the noise is not a defect of the code, it is arithmetic.

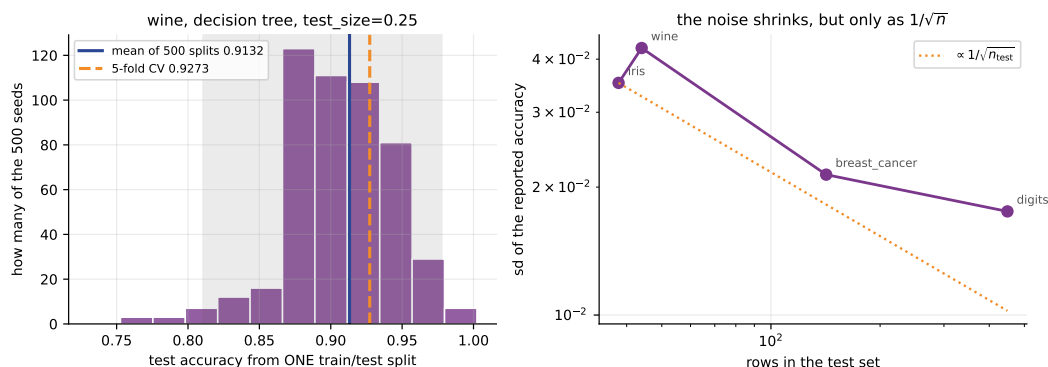


Figure 1: Left: 500 single train/test splits of load_wine, with the shaded band the middle 95%. Right: the same experiment on four bundled datasets. The spread falls as $1/\sqrt{n_{te}}$, so *halving* the noise costs *four times* the test set.

```
# fewer seeds on the bigger datasets, because each seed costs a fit
for nm, X_, y_, n in (("iris", Xi, yi, 500), ("wine", Xw, yw, 500),
                    ("breast_cancer", Xb, yb, 300), ("digits", Xg, yg, 100)):
    a = split_spread(X_, y_, DecisionTreeClassifier(random_state=0), n=n)
```

```
print("%-14s test rows %4d sd %.4f min %.4f max %.4f"
      % (nm, round(0.25*len(y_)), a.std(), a.min(), a.max()))
```

iris	n= 150	test rows	38	sd 0.0352	min 0.7895	max 1.0000
wine	n= 178	test rows	44	sd 0.0425	min 0.7556	max 1.0000
breast_cancer	n= 569	test rows	142	sd 0.0214	min 0.8601	max 0.9790
digits	n=1797	test rows	449	sd 0.0186	min 0.7800	max 0.9111

Roughly three times the test rows, roughly half the standard deviation: 0.0425 at 44 rows against 0.0214 at 142. That is the $1/\sqrt{n}$ law, measured.

2.4 It is not just noisy — it ranks models wrongly

Noise that only blurred the number would be tolerable. The real damage is that a single split *makes decisions*. Take two models that are, on average, indistinguishable: an unpruned decision tree and the same tree limited to depth 3.

```
d = np.empty(500)
for s in range(500):
    Xtr, Xte, ytr, yte = train_test_split(Xw, yw, test_size=0.25, random_state=s)
    d[s] = (clone(m1).fit(Xtr, ytr).score(Xte, yte) # unpruned
           - clone(m2).fit(Xtr, ytr).score(Xte, yte)) # max_depth=3
print("mean difference %.4f sd %.4f" % (d.mean(), d.std()))
print("depth-3 won on %d, unpruned on %d, tied on %d"
      % ((d < 0).sum(), (d > 0).sum(), (d == 0).sum()))
```

```
wine: an unpruned tree against a depth-3 tree, 500 single splits
mean difference -0.0021 sd 0.0276
the depth-3 tree won on 119 splits, the unpruned tree on 74, tied on 307
most extreme single-split verdicts: +0.1556 and -0.1333
```

If your model comparison rests on one split, you measured the split

These two models differ by 0.0021 on average — two-tenths of one per cent. A single split declared a winner on 193 of 500 occasions, and the size of its verdict ranged from +0.1556 to −0.1333. Somebody who ran one seed would write “pruning helps by sixteen points”; somebody who ran another would write the opposite. Both would be reporting noise with a straight face.

The fix is not a luckier seed. It is to **average over many splits**, which is what the rest of this lecture is about. But before averaging, we have to make sure the individual splits are legitimate at all.

3 Split strategies: stratified, grouped, ordered

Every random split makes one assumption:

Key idea

Rows are **independent draws from a single population**, so any random subset is as good as any other. When that assumption fails — and it fails constantly — a random split does not merely add noise, it *leaks*.

Three failures, three repairs:

When it fails	Why	What to use
Skewed classes	a fold may contain few or no minority rows	StratifiedKFold, stratify=y
Repeated measurements	rows from the same patient, sensor, image or customer are near-duplicates	GroupKFold, GroupShuffleSplit, StratifiedGroupKFold
Time order	tomorrow is predicted from yesterday, not from next week	TimeSeriesSplit

The test that covers all three: **could a test row have been produced by something the training set already contains?** If yes, your score measures recall of the training set.

3.1 Stratification, by hand and in code

Stratification is not clever. It deals each class out separately, in proportion. On the twelve-row table of Section 2, three folds:

```
for nm, cv in (("KFold(3, shuffle=False)", KFold(3)),
              ("StratifiedKFold(3)", StratifiedKFold(3))):
    for k, (tr, te) in enumerate(cv.split(X12, y12)):
        print("fold %d test rows %s y %s minority %d"
              % (k+1, te.tolist(), y12[te].tolist(), y12[te].sum()))
```

```
twelve rows, y = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1]
KFold(3, shuffle=False)
  fold 1 test rows [0, 1, 2, 3]    y [0, 0, 0, 0] minority 0
  fold 2 test rows [4, 5, 6, 7]    y [0, 0, 0, 0] minority 0
  fold 3 test rows [8, 9, 10, 11] y [0, 1, 1, 1] minority 3
StratifiedKFold(3)
  fold 1 test rows [0, 1, 2, 9]    y [0, 0, 0, 1] minority 1
  fold 2 test rows [3, 4, 5, 10]   y [0, 0, 0, 1] minority 1
  fold 3 test rows [6, 7, 8, 11]   y [0, 0, 0, 1] minority 1
```

Three ill patients, three folds, one each: exactly the arithmetic of the worked example. Note also that `KFold` *without* `shuffle` takes contiguous blocks of rows, which on a sorted file is the worst available choice — and `shuffle=False` is the default.

3.2 A fold with nothing to learn from

Scale it up: 200 rows, 10 positives (5%), file sorted by label.

```
for nm, cv in (("KFold(5, shuffle=False)", KFold(5)),
              ("KFold(5, shuffle=True, random_state=0)",
               KFold(5, shuffle=True, random_state=0)),
              ("StratifiedKFold(5, shuffle=True, random_state=0)",
               StratifiedKFold(5, shuffle=True, random_state=0))):
    print("%-48s positives per test fold %s"
          % (nm, [int(y_im[te].sum()) for _, te in cv.split(X_im, y_im)]))
```

```
200 rows, 10 positives (5%), and the file is sorted by label
KFold(5, shuffle=False) positives per test fold [10, 0, 0, 0, 0]
KFold(5, shuffle=True, random_state=0) positives per test fold [3, 1, 2, 3, 1]
StratifiedKFold(5, shuffle=True, random_state=0) positives per test fold [2, 2, 2, 2, 2]
```

Look at what `KFold(5, shuffle=False)` has done, in class counts:

```
KFold(5, shuffle=False) class counts, [negatives positives]:
  fold 1 train [160, 0] test [30, 10]
  fold 2 train [150, 10] test [40, 0]
  fold 3 train [150, 10] test [40, 0]
```

```
fold 4  train [150, 10]  test [40, 0]
fold 5  train [150, 10]  test [40, 0]
```

Fold 1's *training set contains one class*. A classifier cannot be fitted at all, so scikit-learn returns `nan` for that fold. Folds 2–5 contain no positives to test on, so each of them scores accuracy 1.0000 and recall 0 vacuously. Cross-validate and read the summary:

```
KFold(5, shuffle=False)      recall  per fold [nan, 0.0, 0.0, 0.0, 0.0]
                             accuracy per fold [nan, 1.0, 1.0, 1.0, 1.0]
                             mean recall 0.0000   mean accuracy 1.0000
KFold(5, shuffle=True)      recall  per fold [0.6667, 1.0, 0.5, 0.3333, 1.0]
                             accuracy per fold [0.975, 1.0, 0.975, 0.95, 1.0]
                             mean recall 0.7000   mean accuracy 0.9800
StratifiedKFold(5, shuffle=True) recall  per fold [1.0, 0.5, 0.5, 0.5, 0.5]
                             accuracy per fold [1.0, 0.975, 0.975, 0.975, 0.975]
                             mean recall 0.6000   mean accuracy 0.9800
```

Mean accuracy 1.0000, mean recall 0.0000, and no error message

The first row is a perfect score produced by a broken experiment. Nothing in the output says so. The only defence is to look at the per-fold numbers and the per-fold class counts — not the mean.

Notice a second, subtler point in that table. Shuffling repaired the catastrophe but gave folds with 1, 2 and 3 positives, and the mean recall it reports (0.7000) is *higher* than the stratified one (0.6000). Higher is not better here: it is an average over folds of unequal difficulty. On the three-class wine data the same effect shows up as a drift in the class mix:

```
KFold(5, shuffle=True, random_state=0)
  class 0 share per fold [0.3889, 0.2222, 0.3333, 0.4286, 0.2857]
  largest deviation from the overall share 0.1132
StratifiedKFold(5, shuffle=True, random_state=0)
  class 0 share per fold [0.3333, 0.3333, 0.3333, 0.3429, 0.3143]
  largest deviation from the overall share 0.0297
overall class shares [0.3315, 0.3989, 0.2697]
```

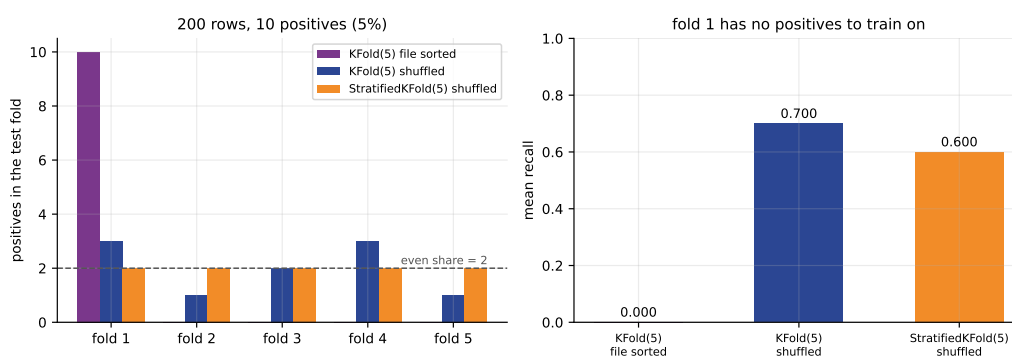


Figure 2: A 5% minority in a sorted file. Left: positives per test fold under three splitters. Right: the mean recall each one reports — 0.000, 0.700, 0.600. Stratifying gives the *lowest* number of the three and the only trustworthy one.

Key idea

For classification, stratify by default. `cross_val_score` already does when you pass an integer `cv` and a classifier; if you construct the splitter yourself, construct `StratifiedKFold(k, shuffle=True, random_state=...)`, not `KFold(k)`.

3.3 Grouped rows: the unit of independence

Here is the experiment that convinces people. Forty patients. Each contributes fifteen windows of a recorded signal, so 600 rows. Each patient’s **diagnosis is a coin flip**, unrelated to anything. Each patient has a random ten-dimensional “fingerprint” — a baseline that says nothing about the diagnosis — and the features are that fingerprint plus noise.

There is nothing to learn. The honest score is 0.5000.

```
pat_sig = rng.normal(size=(40, 10)) * 2.0 # a per-patient fingerprint
y_pat = rng.integers(0, 2, 40) # diagnosis: a coin flip
groups = np.repeat(np.arange(40), 15)
Xp = pat_sig[groups] + rng.normal(size=(600, 10)) * 0.5
yp = y_pat[groups]

knn = make_pipeline(StandardScaler(), KNeighborsClassifier(5))
print(cross_val_score(knn, Xp, yp,
                      cv=StratifiedKFold(5, shuffle=True, random_state=0)).mean())
print(cross_val_score(knn, Xp, yp, groups=groups, cv=GroupKFold(5)).mean())
```

```
random 5-fold, rows shuffled : 1.0000 +/- 0.0000
GroupKFold(5) by patient      : 0.6483 +/- 0.1984
overstatement                 : +0.3517
patients in BOTH train and test, random fold 1 : 37 of 40
patients in BOTH train and test, GroupKFold fold 1: 0
```

1.0000 ± 0.0000 on a coin flip. Thirty-seven of the forty patients had windows on both sides of fold 1, so the nearest neighbour of a test window was another window *from the same patient*, and the model simply read the label off it. The classifier is not diagnosing; it is recognising.

Two further things to read in that output. First, **GroupKFold** gives 0.6483, not 0.5000 — with only forty independent patients, an estimate of a coin flip is itself noisy. Second, and more important, the honest estimate comes with an honest error bar, ± 0.1984 , because you have **forty** independent observations and not six hundred. The random split reported no uncertainty at all, which should always be the first thing you distrust.

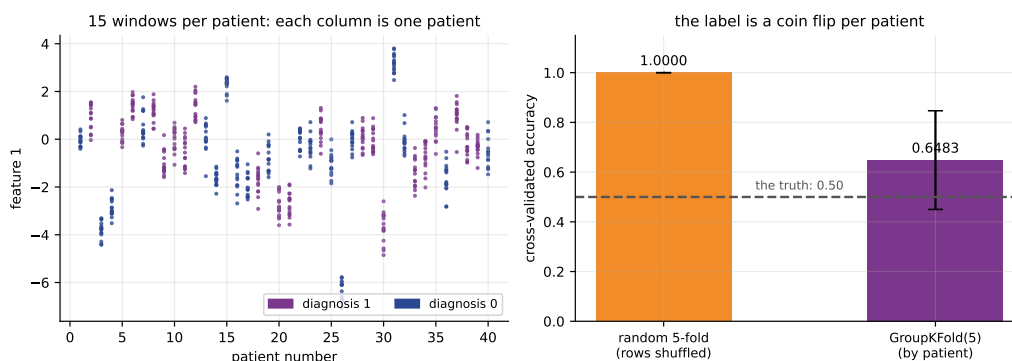


Figure 3: Left: each cluster is one patient’s fifteen windows, projected onto two dimensions — tight, and all carrying the same label. Right: the two verdicts on the same data.

What GroupKFold actually does. When several rows come from one real-world thing — a patient, a customer, a document — that thing is a **group**. **KFold** and **StratifiedKFold** deal out *rows*, so nothing stops one patient’s rows from landing on both sides of a split. **GroupKFold** deals out *groups* instead. You pass one group ID per row through the **groups=** argument; it divides the distinct IDs into k folds, trying to keep the number of rows per fold roughly equal, and every row carrying a given ID goes into the same fold. Each group is therefore tested exactly once, and never while it is also in the training set. On twelve rows from four patients

A–D, three rows each:

```
KFold(4, shuffle=True, random_state=0)
  fold 1  test patients B C D   also in train: B, C, D
  fold 2  test patients A C D   also in train: A, C, D
  fold 3  test patients A C D   also in train: A, C, D
  fold 4  test patients A B B   also in train: A, B
GroupKFold(4)
  fold 1  test patients D D D   also in train: none
  fold 2  test patients C C C   also in train: none
  fold 3  test patients B B B   also in train: none
  fold 4  test patients A A A   also in train: none
```

Two consequences follow. You need at least k groups, since a fold cannot hold less than one. And the effective sample size is the number of groups, not the number of rows — which is exactly why the grouped estimate above carries the wider error bar.

Key idea

Ask “what is the unit of independence?” before you split. It is the patient, not the window; the customer, not the transaction; the document, not the sentence; the subject, not the trial.

Grouping is not the same as stratifying, and you often need both

GroupKFold keeps groups intact but ignores the class balance; **StratifiedKFold** balances classes but ignores groups. When the target is skewed *and* the rows are grouped, use **StratifiedGroupKFold**, which attempts both. Note also that by default **GroupKFold** is deterministic — the same groups always land in the same folds. Since scikit-learn 1.6 it accepts `shuffle=True`, `random_state=...`, so to average over several groupings, repeat it with different seeds (or use **GroupShuffleSplit**).

3.4 Time-ordered rows

Six hundred days of a drifting, seasonal series: a random walk, plus a linear trend, plus a 30-day seasonal cycle, plus noise. The features are the day index and two seasonal terms; the model is a random forest.

```
itr, ite = train_test_split(np.arange(600), test_size=0.25, random_state=0)
r_shuf = clone(rf).fit(Xt[itr], series[itr]).score(Xt[ite], series[ite])
cut = 450
r_time = clone(rf).fit(Xt[:cut], series[:cut]).score(Xt[cut:], series[cut:])
r_tss = cross_val_score(rf, Xt, series, cv=TimeSeriesSplit(5), scoring="r2")
```

```
600 days of a drifting seasonal series; the features are the day
index and two seasonal terms
  random 75/25 split          R2 +0.9975
  train on the first 75% of days R2 -2.7683
  TimeSeriesSplit(5), per fold [-1.8789, -0.0513, 0.0816, -4.1669, -8.0681]
  TimeSeriesSplit(5), mean      R2 -2.8167
  overstatement from shuffling  +3.8142
  in the random split the median gap between consecutive TRAINING
  days is 1: almost every test day sits between two training days
  92 of the 150 test days have BOTH neighbours in the training set (61.3%)
```

$R^2 = +0.9975$ against an honest -2.8167 . This is the most persuasive failure in the lecture, and the mechanism is visible in the last two lines: a random split leaves 61.3% of test days with *both* neighbours in the training set, so the model interpolates between two known values. In production it must **extrapolate**, and a random forest structurally cannot: every prediction it

makes is an average of training targets, so it is bounded by the range of the training targets. When the series keeps climbing, the forest's prediction flattens.

Remember what a negative R^2 means: **you would have done better predicting the training mean for every day.**

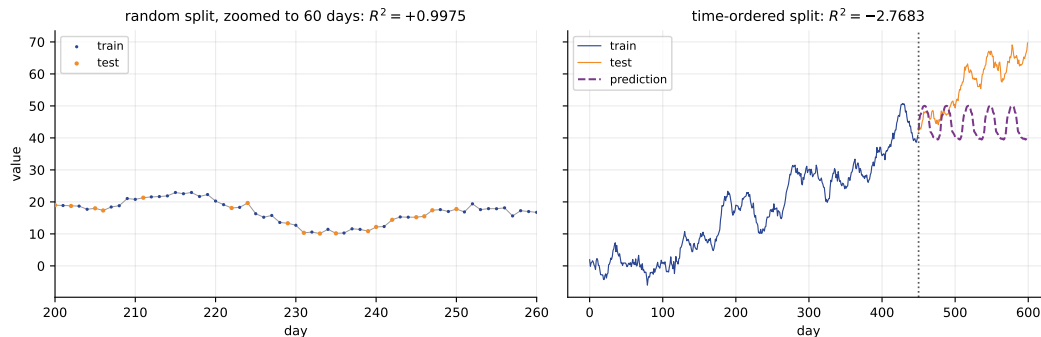


Figure 4: Left, zoomed to sixty days: orange test points sitting between blue training points — the interpolation the random split rewards. Right: trained on the first 450 days, the forest's prediction flattens the moment it leaves the training range.

`TimeSeriesSplit(k)` produces k folds in which the training set is always a prefix of the series and the test set is the block immediately after it. The training set grows from fold to fold; no fold ever trains on the future.

The damage is not confined to the number. Compare two candidate models on the same 600 days — (A) calendar features with a random forest, (B) yesterday's value and a trailing mean with a linear model:

```
A: calendar features + random forest  shuffled 5-fold R2 +0.9974  TimeSeriesSplit R2 -
2.8167
B: lag-1 + trailing mean + linear      shuffled 5-fold R2 +0.9967  TimeSeriesSplit R2
+0.9230
shuffled cross-validation ranks A above B and would ship A.
A cannot extrapolate past the training range at all.
```

The wrong splitter inverts the decision, not just the number

On the shuffled evidence, 0.9974 beats 0.9967 and you ship model A. In time order, A scores -2.82 and B scores $+0.92$. The splitter did not merely inflate a score; it reversed the choice the score existed to make.

Draw your own splitter

Take any `cv` object and print what it actually does:

```
for k, (tr, te) in enumerate(TimeSeriesSplit(4).split(np.zeros((40, 1)))):
    print(k, "train", len(tr), "test", te[:3], "...", te[-1])
```

Expected: training sets of size 8, 16, 24, 32 and test blocks of 8 that never precede their training set. Two minutes of this will tell you more than any diagram — including whether your own data really is grouped.

4 Cross-validation

4.1 The recipe, and its cost

k -fold cross-validation:

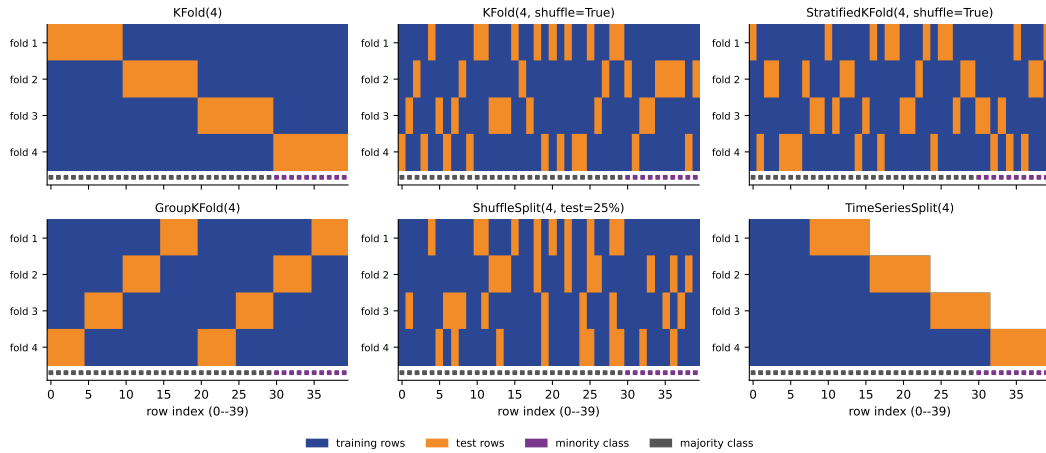


Figure 5: Six splitters on the same forty rows — ten of them minority, arriving sorted, in eight groups of five. Every band was drawn from the indices the splitter actually returned. Minority rows per test fold: KFold [0, 0, 0, 10]; shuffled [0, 6, 2, 2]; stratified [2, 2, 3, 3]; GroupKFold [5, 5, 0, 0]; TimeSeriesSplit [0, 0, 2, 8].

1. Cut the rows into k folds.
2. For each fold in turn: train on the other $k - 1$ folds, test on this one.
3. Report the **mean** of the k scores — and the **spread**.

Every row is tested exactly once, so the estimate is built from all n rows rather than from $n/4$; and you get k numbers instead of one, so you can see how variable the answer is.

The cost model, on $n = 5000$ rows

	fits	rows per fit	row-fits
2-fold	2	2500	5,000
5-fold	5	4000	20,000
10-fold	10	4500	45,000
LOOCV	5000	4999	24,995,000

k -fold costs k times one fit — that is the whole cost model. The row-fits column is $k \cdot n(1 - 1/k) = n(k - 1)$, linear in k , so LOOCV is about n times the work of a single fit on almost all the data.

4.2 Choosing k : bias, variance and time

Three things change as k grows. Measure all three on `load_breast_cancer` with a scaler and a logistic regression.

```
PIPE = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
for nm, cv, nfit in (("2-fold", StratifiedKFold(2, shuffle=True, random_state=0), 2),
                    ...,
                    ("LOOCV", LeaveOneOut(), len(yb))):
    t0 = time.perf_counter()
    s = cross_val_score(PIPE, Xb, yb, cv=cv)
    print("%s %4d fits trains on %3d rows accuracy %.4f %6.2f s"
          % (nm, nfit, len(yb)*(1 - 1/nfit), s.mean(), time.perf_counter()-t0))
```

2-fold	2 fits	trains on 284 rows	accuracy 0.9859	0.01 s
5-fold	5 fits	trains on 455 rows	accuracy 0.9789	0.01 s
10-fold	10 fits	trains on 512 rows	accuracy 0.9772	0.03 s
20-fold	20 fits	trains on 540 rows	accuracy 0.9790	0.06 s
LOOCV	569 fits	trains on 568 rows	accuracy 0.9789	1.62 s

```

LOOCV fits the model 569 times against 5-fold's 5: 113.8 times as many,
and that ratio is the same on every machine
on this run the wall clock made it 112 times, but the seconds above are
a measurement, not a reproducible number
sd ACROSS FOLDS: 5-fold 0.0142    LOOCV 0.1437    <- not comparable

```

Bias. Read the third column, not the fifth. 2-fold trains each model on 284 rows; 20-fold trains it on 540. A model trained on fewer rows is worse, so small k *underestimates* the performance of the model you will actually ship, which is trained on everything. That is the bias of k -fold, and its size is the slope of the learning curve:

```

bias: what a smaller training set costs
train on 50 rows -> held-out accuracy 0.9566
train on 100 rows -> held-out accuracy 0.9666
train on 200 rows -> held-out accuracy 0.9709
train on 300 rows -> held-out accuracy 0.9749
train on 400 rows -> held-out accuracy 0.9757
train on 455 rows -> held-out accuracy 0.9741
train on 512 rows -> held-out accuracy 0.9763
train on 540 rows -> held-out accuracy 0.9776

```

Between 455 rows (what 5-fold trains on) and 540 rows (what 20-fold trains on) the curve moves by about 0.003. On this dataset the bias of 5-fold is therefore small, because 569 rows is already past the steep part of the curve. On 60 rows it would not be: from 50 to 100 rows the curve moves by 0.010, three times as much.

Cost. Read the second column, not the fifth. The cost of k -fold is k fits of the model, so LOOCV on these 569 rows costs 569 fits against 5-fold's 5: 113.8 **times as many, for an identical answer of 0.9789**. That ratio is arithmetic and it is the same on your machine as on mine.

The seconds are a measurement; the fits are the claim

The last column of that table is the one thing in these notes that will not reproduce. It depends on your processor, your BLAS build and what else is running, and it varies between runs *on the same machine*: repeated runs of exactly this comparison gave LOOCV/5-fold wall-clock ratios of 112, 122, 126 and 131 while the fit ratio stayed at 113.8 throughout. So quote the fits and never the seconds. If you time it and get a different ratio, you are not wrong — and neither is the person next to you who got another one. When you need a defensible cost claim, count model fits.

cross_val_score(...).std() is not the uncertainty of the mean

In the table above, the across-fold sd is 0.0142 for 5-fold and 0.1437 for LOOCV. That does *not* mean LOOCV is ten times less stable. Each LOOCV fold scores a single row, so its score is 0 or 1 and the across-fold sd is just $\sqrt{p(1-p)} \approx \sqrt{0.979 \times 0.021} = 0.143$. It is a property of the scoring, not of the estimator. The quantity you actually care about is the spread of the *mean*, which needs repetition to measure.

4.3 Variance of the estimate, measured properly

Repeat the whole cross-validation with 40 different shuffles and look at how much the reported mean moves.

```

for k in (2, 5, 10, 20):
    means = np.array([cross_val_score(
        PIPE, Xb, yb,
        cv=StratifiedKFold(k, shuffle=True, random_state=s)).mean()

```

```
for s in range(40)]])
print("%2d-fold mean of means %.4f sd of the mean %.4f range %.4f"
      % (k, means.mean(), means.std(), means.max() - means.min()))
```

```
spread of the cross-validated MEAN over 40 different shuffles
 2-fold   mean of means 0.9754   sd of the mean 0.0042   range 0.0194
 5-fold   mean of means 0.9782   sd of the mean 0.0030   range 0.0123
10-fold   mean of means 0.9786   sd of the mean 0.0025   range 0.0106
20-fold   mean of means 0.9791   sd of the mean 0.0019   range 0.0072
LOOCV     mean           0.9789   sd of the mean 0.0000   (no shuffle: LOOCV is
deterministic)
one 80/20 split, 40 seeds: mean 0.9772   sd 0.0114   range 0.0526
```

Two results, one of them uncomfortable.

The comfortable one: **cross-validation is about four times steadier than a single split**, for five times the compute. 5-fold has sd 0.0030 where one 80/20 split has 0.0114.

The uncomfortable one: on this dataset the variance *falls* as k rises, and LOOCV has none at all — it does not shuffle, so re-running it changes nothing. Textbooks routinely say “LOOCV has high variance”. That claim is about the variance across *different datasets* drawn from the same population, driven by the near-total overlap of LOOCV’s training sets, and it is not what this experiment measures. Report the experiment you ran, not the one the textbook predicted. What LOOCV certainly is here is a hundred times slower for the same answer.

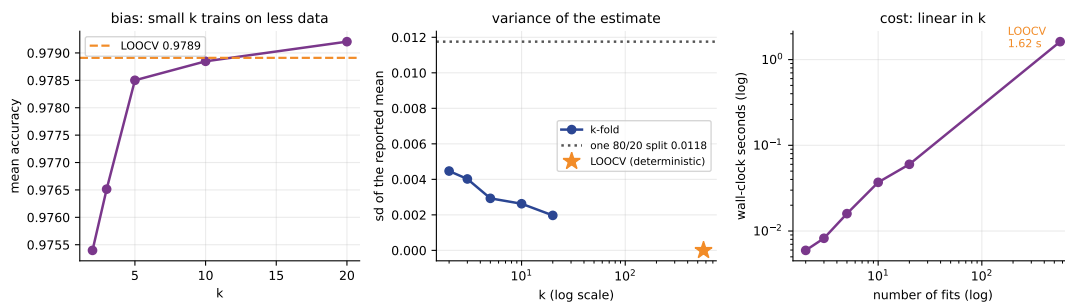


Figure 6: Left: the estimate rises with k and flattens — the bias going away. Middle: the spread of the estimate falls with k ; the dotted line is a single 80/20 split. Right: cost is exactly linear in the number of fits. Five or ten folds is where the three curves meet.

Key idea

Use $k = 5$ or $k = 10$. Use more folds when n is small and a fit is cheap; use fewer when a fit is expensive. Do not use LOOCV because it sounds thorough.

4.4 More folds, or more repeats?

If 5-fold is still too wobbly, you can repeat it with different shuffles rather than increasing k .

StratifiedKFold(5)	5 fits	accuracy 0.9789 +/- 0.0142	0.02 s
RepeatedStratifiedKFold(5 x 10)	50 fits	accuracy 0.9772 +/- 0.0147	0.18 s
StratifiedShuffleSplit(50, test 20%)	50 fits	accuracy 0.9767 +/- 0.0122	0.17 s

StratifiedKFold(5)	the default for classification. Use it.
KFold(5, shuffle=True)	regression, or classification with a balanced target.
RepeatedStratifiedKFold	when n is small and you need the estimate to stop wobbling before you can compare two models.
ShuffleSplit, StratifiedShuffleSplit	when you want the test size and the number of repeats to be independent of each other. Rows may appear in several test sets, and some in none.
LeaveOneOut	when n is tiny and a fit is cheap.
GroupKFold, StratifiedGroupKFold, TimeSeriesSplit	whenever the rows are not independent. Not optional.

Report a score the way a reviewer will accept

```
cv = StratifiedKFold(5, shuffle=True, random_state=0)
s = cross_val_score(PIPE, Xb, yb, cv=cv)
print("accuracy %.4f +/- %.4f over %d folds" % (s.mean(), s.std(), len(s)))
```

Expected: accuracy 0.9789 +/- 0.0142 over 5 folds. Never report the first number without the second, and never report either without saying which splitter produced them.

5 Tuning honestly: validation sets and nested CV

5.1 The moment you tune, the score stops being an estimate

Split `breast_cancer` three ways — train, validation, test — and choose C for a logistic regression on the validation set.

```
the three-way split, done once, on breast_cancer
train 341  validation 114  test 114
C= 0.01  validation 0.9561
C= 0.10  validation 0.9561
C= 1.00  validation 0.9474
C= 10.00 validation 0.9474
C=100.00 validation 0.9298
chosen C=0.01  validation 0.9561  TEST 0.9386
```

0.9561 is the *maximum of five noisy numbers*. It is biased upward by construction, for the same reason that the tallest of five randomly chosen people is taller than average. 0.9386 is the estimate.

Key idea

The validation set **selects**. The test set **estimates**. A number that was used to choose something can never also be the estimate of what was chosen — and you may look at the test set once.

5.2 Predict the number first: tuning on pure noise

Cross-validation replaces the validation *set*, but it does not remove the need for a validation *step*. If the tuning happens inside the same folds you report, the bias comes straight back. Here is how much.

One hundred rows, two hundred features, everything drawn from $\mathcal{N}(0, 1)$. The labels are coin flips — fifty zeros and fifty ones, permuted. **There is no signal whatsoever; the truth is 0.5000.** Tune an RBF SVM over five values of C and four of γ with 5-fold cross-validation, and report `GridSearchCV.best_score_`.

Write down what you expect before reading on.

```
def nested_experiment(X, y, grid, est, trials=20, inner=5, outer=5):
    non_nested, nested = [], []
    for t in range(trials):
        icv = StratifiedKFold(inner, shuffle=True, random_state=t)
        ocv = StratifiedKFold(outer, shuffle=True, random_state=t)
        gs = GridSearchCV(est, grid, cv=icv).fit(X, y)
        non_nested.append(gs.best_score_) # the wrong way
        nested.append(cross_val_score(GridSearchCV(est, grid, cv=icv),
                                       X, y, cv=ocv).mean()) # the right way
    return np.array(non_nested), np.array(nested)
```

```
pure noise, 100 rows x 200 features, coin-flip labels (the truth is 0.5000)
tune and report on the SAME folds : 0.5820 +/- 0.0314
nested cross-validation           : 0.5265 +/- 0.0419
optimism                         : +0.0555
the wrong way beat 0.5 on 20 of 20 trials
grid: 20 combinations, so 100 inner fits per trial
```

Eight points of accuracy above the truth, conjured out of nothing, on twenty trials out of twenty. The mechanism is not subtle: `best_score_` is the **maximum** of twenty cross-validated numbers, each carrying a standard error of a few points, and the maximum of twenty noisy numbers is larger than their mean. If each of m combinations has true score μ and independent noise of sd σ , the expected maximum is roughly $\mu + \sigma\sqrt{2\ln m}$ — it grows with the size of the grid, slowly but without limit.

best_score_ is a selection, not an estimate

It is the score of the winner, measured on the same folds that crowned it. It must never be reported as performance. Report either a nested cross-validation, or the score of the tuned model on a test set that the search never touched.

5.3 The same effect where there is real signal

Noise is a demonstration; here is the everyday case. A 120-row subsample of `breast_cancer` — real signal, real structure — with the same grid and twenty trials.

```
breast_cancer, a 120-row subsample -- real signal this time
tune and report on the SAME folds : 0.9529 +/- 0.0054
nested cross-validation           : 0.9317 +/- 0.0094
optimism                         : +0.0212
paired t over 20 trials: t = 9.58, p = 1.1e-08
the wrong way was higher on 19 of 20 trials
```

Two points of accuracy — and two points is very often the entire claimed improvement in a student project. The paired t -test over the twenty trials gives $p = 1.1 \times 10^{-8}$: this is a systematic bias, not a run of luck.

And the size of the bias is set by how hard you searched:

```
the more hyper-parameter combinations you try, the luckier you get
(pure noise again, so every one of these should be 0.5000)
1 combinations   same-fold 0.5467   nested 0.5467   optimism +0.0000
4 combinations   same-fold 0.5767   nested 0.5567   optimism +0.0200
20 combinations  same-fold 0.5833   nested 0.5217   optimism +0.0617
60 combinations  same-fold 0.5850   nested 0.5067   optimism +0.0783
```

The first row is the proof of the mechanism. With one combination in the grid there is nothing to choose between, so there is nothing to over-fit, and the optimism is **exactly zero** — the two

numbers are identical to four decimal places. Every point of optimism in the rows below it is the price of the search.

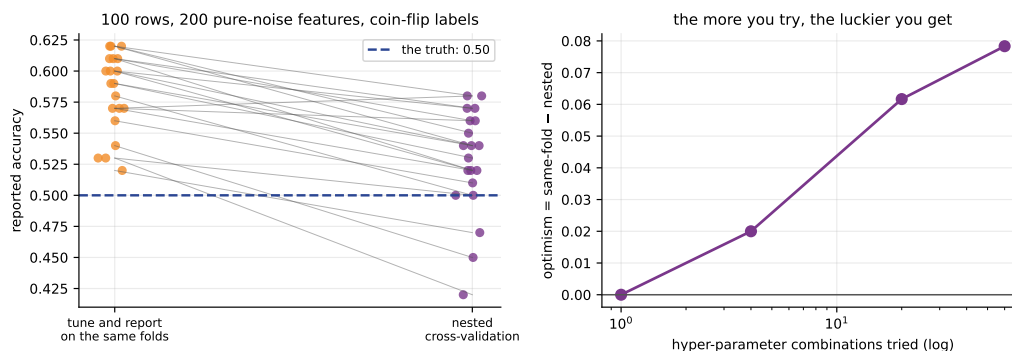


Figure 7: Left: twenty paired trials on pure noise; each grey line joins one trial’s two estimates. Every same-fold point is above 0.50; the nested points scatter around it as they should. Right: optimism against grid size, from +0.0000 at one combination to +0.0783 at sixty.

5.4 Nested cross-validation

```
inner = StratifiedKFold(5, shuffle=True, random_state=0)
outer = StratifiedKFold(5, shuffle=True, random_state=0)
search = GridSearchCV(pipe, grid, cv=inner) # chooses hyper-parameters
score = cross_val_score(search, X, y, cv=outer) # estimates performance
print("%.4f +/- %.4f" % (score.mean(), score.std()))
```

The inner loop **selects**; the outer loop **estimates**. The outer test fold is never involved in choosing anything: not the scaler’s mean, not the feature subset, not C , not γ . Each outer fold may choose a different C , and that is correct — what you are estimating is the performance of *the whole procedure*, tuning included, not of one particular C .

What nested CV costs, and when it is worth it

With a 20-point grid, 5 inner folds and 5 outer folds, one nested estimate costs $5 \times 5 \times 20 = 500$ fits, against 100 for a plain `GridSearchCV`. Five times the compute.

Use nested CV when the number you are about to write down is a **performance claim**.

Use plain `GridSearchCV` when you are **choosing a model** and will evaluate the winner later on a held-out test set. Both are correct. Quoting `best_score_` as your result is not.

6 Imbalanced data

6.1 By hand: one thousand patients, ten of them ill

A model that always answers “healthy”:

	predicted healthy	predicted ill
actually healthy	TN = 990	FP = 0
actually ill	FN = 10	TP = 0

The four numbers

$$\begin{aligned}\text{accuracy} &= \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = \frac{0 + 990}{1000} = \mathbf{0.990}, \\ \text{recall} &= \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{0}{0 + 10} = \mathbf{0}, \\ \text{precision} &= \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{0}{0} \text{ undefined (reported as 0),} \\ \text{balanced accuracy} &= \frac{1}{2} \left(\frac{\text{TN}}{\text{TN} + \text{FP}} + \frac{\text{TP}}{\text{TP} + \text{FN}} \right) = \frac{1}{2}(1 + 0) = \mathbf{0.500}.\end{aligned}$$

Ninety-nine per cent accurate, and it has never once found an ill patient. When TN is 99% of the table, accuracy is a report on TN.

Key idea

Recall asks about the row you care about — of the ill patients, how many did we find? **Precision** asks about the column you act on — of the people we called ill, how many were? F_1 is their harmonic mean. **PR-AUC** (average precision) summarises the whole precision–recall curve, and its chance baseline is the positive rate, not 0.5.

6.2 The same thing measured

Five thousand rows, fifty positives (1%), twenty features of which six are informative.

```
make_classification: 5000 rows, 50 positives (1.00%), ratio 99:1
DummyClassifier(most_frequent) -- it always says 'negative'
accuracy 0.9900
precision 0.0000 recall 0.0000 F1 0.0000
balanced accuracy 0.5000 PR-AUC 0.0100
confusion matrix [[TN FP][FN TP]] = [[1485, 0], [15, 0]]
```

Now a real logistic regression on the same split:

```
plain LogisticRegression on the same split
accuracy 0.9867 (the dummy got 0.9900)
precision 0.0000 recall 0.0000 F1 0.0000
balanced accuracy 0.4983 ROC-AUC 0.8553 PR-AUC 0.1075
confusion matrix [[1480, 5], [15, 0]]
accuracy says the model is worth -0.0033 over the dummy
PR-AUC says it is worth +0.0975
```

The dummy wins on accuracy

The real model has learned a great deal — ROC-AUC 0.8553, PR-AUC nearly eleven times the chance baseline of 0.0100. At the default threshold of 0.5 it still labels almost everything negative, so accuracy rates it *below* a model that has learned nothing at all. Two separate failures are visible here: the metric is wrong, and the threshold is wrong.

How bad does the imbalance have to be before this bites? There is no threshold; it is a slope. Sweep the minority fraction from 50% to 0.2%:

```
imbalance ratio against what accuracy is worth
minority 50.0% (2500 rows) always-negative 0.5000 model accuracy 0.8586 recall 0.8704
PR-AUC 0.9063
minority 20.0% (1000 rows) always-negative 0.8000 model accuracy 0.8874 recall 0.6210
PR-AUC 0.7532
minority 10.0% ( 500 rows) always-negative 0.9000 model accuracy 0.9260 recall 0.4400
PR-AUC 0.6098
```

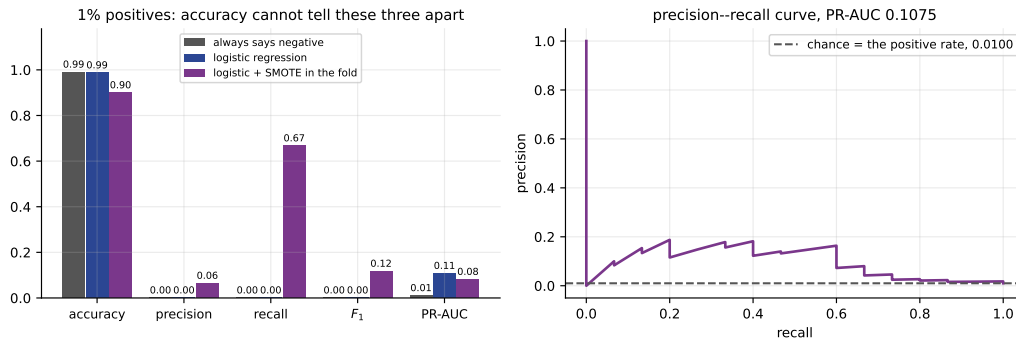


Figure 8: Left: five metrics for three models — always-negative, plain logistic regression, and logistic regression with SMOTE. Accuracy is 0.99, 0.99, 0.90: the one metric that barely distinguishes them is the one everybody quotes. Right: the precision–recall curve, whose baseline is the positive rate 0.01.

minority	5.0% (250 rows)	always-negative	0.9500	model accuracy	0.9544	recall	0.2720
		PR-AUC	0.4392				
minority	1.0% (50 rows)	always-negative	0.9900	model accuracy	0.9890	recall	0.0400
		PR-AUC	0.1916				
minority	0.2% (10 rows)	always-negative	0.9980	model accuracy	0.9980	recall	0.0000
		PR-AUC	0.0277				

Two columns moving in opposite directions. As the problem gets harder, **accuracy goes up** — from 0.86 at balance to 0.998 at 0.2% — while recall falls to zero. Reported accuracy on an imbalanced problem is largely a measure of how imbalanced the problem is.

6.3 Four things you can do about it

	What it does	What it costs
Random oversampling	copies minority rows, with replacement, until the classes balance	exact duplicates: the model can memorise them
Random undersampling	throws majority rows away until the classes balance	discards most of your data
SMOTE	invents minority rows on the segments between existing ones	interpolates into regions where nothing was observed
Class weights	multiplies the loss of minority errors; no rows added or removed	nothing, and it is one keyword argument

And the one that is not on the list: **move the threshold**. A classifier gives you a probability; 0.5 is a convention, not a result. Choosing the operating point from the precision–recall curve costs nothing and is often all that is needed.

Written out, with no dependency on `imbalanced-learn`:

```
rng = np.random.default_rng(0) # the one seed for this lecture; every
                                # resampler below is handed this rng

def random_oversample(X, y, rng):
    out_X, out_y = [X], [y]
    n_max = np.bincount(y).max()
    for c in np.unique(y):
        idx = np.where(y == c)[0]
        need = n_max - len(idx)
        if need > 0:
            pick = rng.choice(idx, size=need, replace=True)
```

```

        out_X.append(X[pick]); out_y.append(y[pick])
    return np.vstack(out_X), np.concatenate(out_y)

def random_undersample(X, y, rng):
    n_min = np.bincount(y).min()
    keep = np.concatenate([rng.choice(np.where(y == c)[0], size=n_min,
                                      replace=False) for c in np.unique(y)])
    rng.shuffle(keep)
    return X[keep], y[keep]

```

6.4 SMOTE is one line of arithmetic

SMOTE between two patients

Two minority patients, $A = (1, 1)$ and $B = (3, 5)$. SMOTE draws $u \sim \mathcal{U}(0, 1)$ and returns the point $A + u(B - A)$:

$$u = 0 \rightarrow (1, 1) = A, \quad u = \frac{1}{4} \rightarrow (1.5, 2), \quad u = \frac{1}{2} \rightarrow (2, 3), \quad u = 1 \rightarrow (3, 5) = B.$$

Check the middle one by hand: $B - A = (2, 4)$, so $A + \frac{1}{2}(2, 4) = (1, 1) + (1, 2) = (2, 3)$. That is the whole idea. Every synthetic point lies on a straight segment between a real minority point and one of its k nearest minority neighbours.

SMOTE by hand between $A = (1, 1)$ and $B = (3, 5)$

```

u = 0.00 -> A + u(B - A) = [1.0, 1.0]
u = 0.25 -> A + u(B - A) = [1.5, 2.0]
u = 0.50 -> A + u(B - A) = [2.0, 3.0]
u = 1.00 -> A + u(B - A) = [3.0, 5.0]

```

The whole of Chawla et al. (2002), in fifteen lines:

```

def smote(X, y, rng, k=5, minority=1):
    Xmin = X[y == minority]
    n_need = int((y != minority).sum() - len(Xmin))
    if n_need <= 0:
        return X, y
    nn = NearestNeighbors(n_neighbors=min(k + 1, len(Xmin))).fit(Xmin)
    _, ind = nn.kneighbors(Xmin)
    ind = ind[:, 1:] # drop the point itself
    base_i = rng.integers(0, len(Xmin), n_need)
    nbr_i = ind[base_i, rng.integers(0, ind.shape[1], n_need)]
    u = rng.random((n_need, 1))
    new = Xmin[base_i] + u * (Xmin[nbr_i] - Xmin[base_i])
    return np.vstack([X, new]), np.concatenate([y, np.full(n_need, minority)])

```

Key idea

SMOTE is not generating data. It is **asserting** that the straight line between two ill patients is full of ill patients. Sometimes that is a reasonable assumption. Sometimes — categorical features, multi-modal classes, minority points sitting inside a majority region — it is exactly wrong.

What the three resamplers do to the 1% problem, on a 3500-row training set:

```

random oversample  train 3500 -> 6930 rows, class counts [3465, 3465]
random undersample train 3500 -> 70 rows, class counts [35, 35]
SMOTE              train 3500 -> 6930 rows, class counts [3465, 3465]
                    3430 of the 3465 minority rows are synthetic (99.0%)
                    distinct minority rows: SMOTE 3465, oversampling 35

```

Read the last line twice. Oversampling turned 35 patients into 3465 rows that are copies of 35 **distinct patients**. SMOTE turned them into 3465 distinct points, but every one of them lies on a segment between two of the same 35. **Neither method has added a single new observation.** Undersampling, meanwhile, kept 70 of 3500 rows — 99.0% of the negatives thrown away.

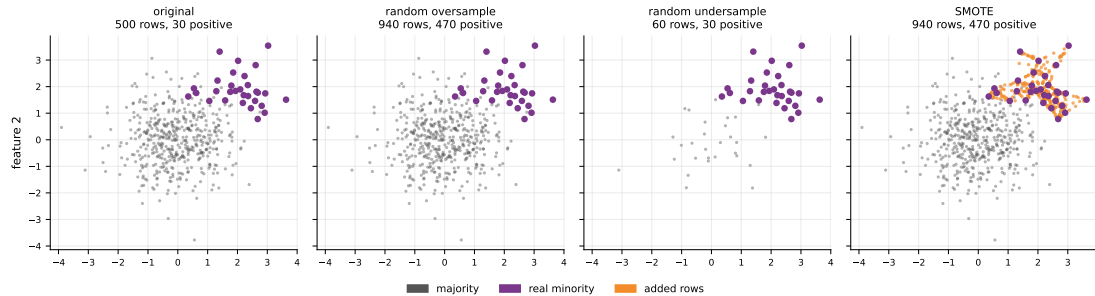


Figure 9: A two-dimensional version so you can see it. Purple is real minority, orange is added. Oversampling appears to add nothing — the new points are exactly on top of the old ones. SMOTE’s additions trace the segments between neighbours, which is why the cloud looks like a web. Undersampling simply deletes most of the majority class.

6.5 Does any of it help?

Five-fold stratified cross-validation, resampling applied **inside** each fold to the training part only, logistic regression.

```
def cv_imbalanced(X, y, make_model, resampler=None, k=5, seed=0):
    cv = StratifiedKFold(k, shuffle=True, random_state=seed)
    acc, pre, rec, f1s, ap = [], [], [], [], []
    for tr, te in cv.split(X, y):
        Xa, ya = X[tr], y[tr]
        if resampler is not None: # <-- INSIDE the fold,
            Xa, ya = resampler(Xa, ya, rng) # training part only
        m = make_model().fit(Xa, ya)
        p, s = m.predict(X[te]), m.predict_proba(X[te])[:, 1]
        acc.append(accuracy_score(y[te], p)); pre.append(precision_score(y[te], p))
        rec.append(recall_score(y[te], p)); f1s.append(f1_score(y[te], p))
        ap.append(average_precision_score(y[te], s))
    return [float(np.mean(v)) for v in (acc, pre, rec, f1s, ap)]
```

method	accuracy	precision	recall	F1	PR-AUC
always negative	0.9900	0.0000	0.0000	0.0000	0.0100
no treatment	0.9890	0.1333	0.0400	0.0583	0.1916
random oversample	0.8542	0.0517	0.7800	0.0969	0.1369
random undersample	0.7944	0.0390	0.8200	0.0744	0.1142
SMOTE	0.8814	0.0612	0.7600	0.1133	0.1404
class_weight balanced	0.8556	0.0520	0.7800	0.0974	0.1368

Three things to take from this table, and the third one is the honest one.

First, recall moves enormously: 0.0400 with no treatment, 0.76 to 0.82 with any of the four. If your problem is “find the ill patients”, that is the difference between a useless model and a usable one.

Second, `class_weight="balanced"` matches random oversampling almost exactly — recall 0.7800 in both, accuracy 0.8556 against 0.8542, F_1 0.0974 against 0.0969. That is as it should be: weighting a row by w and including w copies of it are the same thing to a loss function that is a sum over rows. Prefer the class weight: no rows to manage, no randomness, no opportunity to leak.

Third, and this is the result to report rather than hide: none of the four treatments improved PR-AUC. The best PR-AUC in the table, 0.1916, belongs to the untreated model. Resampling moved the decision threshold; it did not create information. The ranking of the test rows by predicted probability — which is what PR-AUC measures — got slightly *worse*.

Resampling is not free, and it is not a substitute for a threshold

Everything the resamplers achieved on recall could have been achieved by lowering the decision threshold on the untreated model, which is cheaper, has no randomness in it, and leaves the probability estimates calibrated. Try that first, and use the precision–recall curve to choose the operating point.

The same table with a random forest instead is a useful corrective:

method	accuracy	precision	recall	F1	PR-AUC
no treatment	0.9900	0.0000	0.0000	0.0000	0.2564
random oversample	0.9900	0.1000	0.0200	0.0333	0.1981
random undersample	0.7558	0.0359	0.9000	0.0691	0.1105
SMOTE	0.9876	0.2000	0.0800	0.1083	0.2056
class_weight balanced	0.9900	0.0000	0.0000	0.0000	0.2769

The forest ranks better than the logistic regression did (PR-AUC 0.2564 untreated, the best number anywhere in this lecture), and every resampler except `class_weight` made its ranking worse. Note also that `class_weight="balanced"` left the forest's *predictions* unchanged — recall 0.0000 with and without — because for a tree ensemble the weight enters the split criterion rather than a smooth additive loss, and the majority vote at the leaves need not move at all. **Different model, different answer. Measure; do not assume.**

6.6 The mistake: resampling before the split

Everything above resampled inside the loop. Here is what happens when the same line is written one level higher — `X, y = smote(X, y)` before `cross_val_score` instead of inside it. Nothing else changes: same data, same model, same five folds.

```
# WRONG
rng = np.random.default_rng(0) # the one seed
Xall, yall = smote(Xim, yim, rng) # the whole dataset
f1_bad = cross_val_score(mk(), Xall, yall, cv=cv, scoring="f1").mean()

# RIGHT: see cv_imbalanced above -- resample inside the loop
```

the mistake: resample the WHOLE dataset, then cross-validate	
random oversample BEFORE the split :	F1 0.8607 recall 0.8655 PR-AUC 0.9176
random oversample INSIDE each fold :	F1 0.0969 recall 0.7800 PR-AUC 0.1369
random oversample overstatement in F1	+0.7638
SMOTE BEFORE the split :	F1 0.8942 recall 0.9067 PR-AUC 0.9207
SMOTE INSIDE each fold :	F1 0.1133 recall 0.7600 PR-AUC 0.1404
SMOTE overstatement in F1	+0.7809

$F_1 = 0.8942$ against an honest 0.1133. An eightfold overstatement, from moving one line of code. And it is exactly the kind of number that makes a report look finished, which is why it survives to submission.

Why does it happen? Look at what is in the test fold afterwards:

```
after oversampling before the split, fold 1 has 1980 test rows
of which 990 (50.0%) are byte-identical to a training row
among the 990 MINORITY test rows, 990 (100.0%) are copies
```

Every single minority row in the test fold is a byte-identical copy of a row the model trained on. The model is not being tested; it is being asked to remember. SMOTE is only slightly less blatant — its synthetic points are built *from* the training points, using their coordinates and their neighbourhood structure, so the same information crosses the boundary in a less obvious form.

There is a second, quieter problem in that experiment even setting leakage aside: after resampling the whole dataset the test folds are 50/50 balanced. That is not the balance you will meet in production, so even a leak-free score computed on such a fold answers the wrong question.

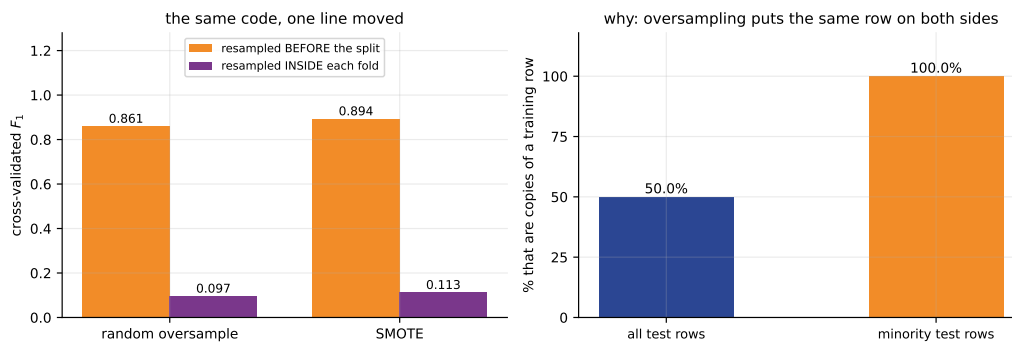


Figure 10: Left: F_1 for both resamplers, before the split and inside the fold. Right: the mechanism — half of every test fold, and *all* of its minority half, are duplicates of training rows.

The rule, stated once

Resampling is part of the model. It happens inside the fold, on the training part only. The test fold keeps its natural class balance, because that is the balance you will meet in production.

This is the same rule as Lecture 6’s rule about scalers and feature selection, and it has the same one-line test: *did this step look at the data?* If yes, it belongs inside the loop. `imbalanced-learn`’s `imblearn.pipeline.Pipeline` exists precisely because `scikit-learn`’s own `Pipeline` cannot hold a step that changes the number of rows; if you resample by hand, you must place the call yourself, and this is where people put it in the wrong place.

6.7 The whole thing in one place

Split, tune on the training part only, resample inside each inner fold, refit, and report once on the test set.

```
Xtr, Xte, ytr, yte = train_test_split(Xim, yim, test_size=0.3,
                                     stratify=yim, random_state=0)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
best = (-1.0, None)
for C in [0.01, 0.1, 1, 10]:
    f1v = []
    for tr, va in cv.split(Xtr, ytr):
        Xa, ya = smote(Xtr[tr], ytr[tr], rng) # inside the fold
        m = make_pipeline(StandardScaler(),
                          LogisticRegression(C=C, max_iter=5000)).fit(Xa, ya)
        f1v.append(f1_score(ytr[va], m.predict(Xtr[va]))) # va NOT resampled
    if np.mean(f1v) > best[0]:
        best = (float(np.mean(f1v)), C)
Xa, ya = smote(Xtr, ytr, rng) # refit on all training rows
final = make_pipeline(StandardScaler(),
                      LogisticRegression(C=best[1], max_iter=5000)).fit(Xa, ya)
```

```

the whole thing in one place: split, tune, resample, evaluate
C= 0.01 inner F1 0.1271
C= 0.10 inner F1 0.1236
C= 1.00 inner F1 0.1215
C= 10.00 inner F1 0.1215
chosen C=0.01 (inner F1 0.1271)
held-out TEST: accuracy 0.8940 precision 0.0610 recall 0.6667 F1 0.1117 PR-AUC 0.0842
confusion matrix [[1331, 154], [5, 10]]

```

Read the confusion matrix, not the decimals: **ten of the fifteen ill patients found, at the cost of 154 false alarms out of 1485 healthy people.** Whether that trade is acceptable is a clinical decision, not a modelling one, and the most useful thing a modeller can do is state it in those terms.

Break it deliberately

Take the block above and move the `smote` call to the line before `train_test_split`. Everything else stays. You should see the held-out F_1 leap from about 0.11 to about 0.89 — and then convince yourself, by checking for duplicate rows across the split, that the leap is entirely fictitious. Doing this once, deliberately, is the best inoculation available.

7 Summary

1. A single train/test score is a **random variable**. On wine, 500 seeds gave 0.7556 to 1.0000; the middle 95% spans 17 points.
2. That noise reverses model comparisons. Two trees differing by 0.0021 on average produced single-split verdicts from +0.1556 to −0.1333, and a “winner” on 193 of 500 splits.
3. Test-set noise falls only as $1/\sqrt{n_{te}}$: sd 0.0425 on 44 test rows against 0.0214 on 142.
4. **Stratify**. On a sorted 5% file, `KFold` put all ten positives in fold 1, left fold 1’s training set with one class, and reported mean accuracy 1.0000 with mean recall 0.0000.
5. **Group**. Forty patients with coin-flip labels scored 1.0000 ± 0.0000 under a random split and 0.6483 ± 0.1984 under `GroupKFold`, because 37 of 40 patients appeared on both sides of fold 1.
6. **Respect time**. A random split scored $R^2 = +0.9975$ where the honest time-ordered answer was −2.8167, and shuffled cross-validation ranked a useless model above a good one.
7. k -fold costs k fits. On 569 rows, LOOCV costs 569 fits against 5-fold’s 5 — $113.8\times$ as many, on any machine — to reach the identical 0.9789.
8. Small k is pessimistic because it starves the model: 2-fold trains on 284 rows, 20-fold on 540, and the learning curve between them is worth about 0.003 here.
9. `GridSearchCV.best_score_` is a **maximum**, not an estimate. On pure noise it read 0.5820 against a nested 0.5265; with a one-point grid the gap was exactly zero; with sixty points it was +0.0783.
10. At 1% positives, always-say-no scores 0.9900 and a real model scores 0.9867: **accuracy ranked the useless model higher**.
11. Oversampling, undersampling, SMOTE and class weights all took recall from 0.0400 to about 0.78 — and **none improved PR-AUC**. They move the threshold; they do not create

information.

12. **Resampling before the split reported $F_1 = 0.8942$ against an honest 0.1133**, because 100% of the minority test rows were copies of training rows.

One sentence for the whole of Unit IV

Anything you learn from the data — a mean, a category list, a feature subset, a hyper-parameter, a resampled row — is part of the model, and belongs inside the loop.

8 Practice questions

1. **Folds by hand.** A dataset has 20 rows, of which rows 17, 18 and 19 (zero-indexed) are the only positives; the file is sorted. Write down the five test folds produced by `KFold(5)` and by `StratifiedKFold(5)`, and the number of positives in each. What goes wrong with the `KFold` version?
2. **A confusion matrix.** A fraud detector on 10,000 transactions produces $TN = 9900$, $FP = 60$, $FN = 30$, $TP = 10$. Compute accuracy, precision, recall, F_1 and balanced accuracy. What accuracy would a model that always predicts “not fraud” achieve? Which single number would you put in the report, and why?
3. **Cost.** For $n = 5000$ rows, compute the number of fits and the total number of row-fits for 2-fold, 5-fold, 10-fold and LOOCV. If one fit on 4000 rows takes 0.4 seconds and cost scales linearly with rows, how long does each take?
4. **Reporting.** Five hundred random 75/25 splits of `load_wine` gave a 2.5th–97.5th percentile range of $[0.8106, 0.9778]$. A classmate reports “our model achieves 97.8% accuracy”. Write the two-sentence reply you would give in a review, and state what you would ask them to run instead.
5. **Choose the splitter.** For each of the following, name the splitter and give the reason in one line. (a) 12,000 chest X-rays from 3000 patients, predicting pneumonia. (b) Daily electricity demand for a campus, predicting tomorrow. (c) 600 wine samples, three cultivars, roughly balanced. (d) 200,000 card transactions, 0.17% fraudulent, from 40,000 cardholders, with a timestamp on every row. (e) 60 students’ examination scripts marked by six examiners, with the examiner recorded.
6. **SMOTE by hand.** Minority points $A = (2, 4)$ and $B = (6, 10)$, with B the nearest minority neighbour of A . Compute the synthetic point for $u = 0.3$. Now suppose a third feature is “blood group”, one-hot encoded as $(1, 0, 0)$ for A and $(0, 1, 0)$ for B . What does SMOTE produce for that block at $u = 0.3$, and what is wrong with it?
7. **Why `best_score_` is biased.** A grid has m combinations. Every one of them has the same true accuracy μ , and the cross-validated estimate of each is $\mu + \varepsilon_i$ with the ε_i independent and mean zero. Explain in three sentences why $\mathbb{E}[\max_i(\mu + \varepsilon_i)] > \mu$, and predict what happens to the bias as m grows from 1 to 60. Check your prediction against the table in Section 5.2.
8. **Debug this.** A student reports: “I have 2% churners. I applied SMOTE to balance the dataset, then used `cross_val_score` with 5-fold and got $F_1 = 0.89$. The model is ready to deploy.” Identify every error, say what the honest number is likely to look like, and write the corrected procedure as six numbered steps.
9. **Debug this too.** A student predicting hourly air quality reports $R^2 = 0.998$ using `KFold(5, shuffle=True)` on three years of hourly readings, with features “hour of day”, “day of year”

and “station id”. Explain the two independent problems with the evaluation, and describe the experiment that would show the score is an illusion.

10. **Weights or copies?** `class_weight="balanced"` and random oversampling gave nearly identical results for logistic regression in Section 6.5, but visibly different results for a random forest. Explain both observations. Which would you use in production, and what would you check before deciding?

9 Solutions

Solution 1. By hand: `KFold(5)` on 20 sorted rows takes contiguous blocks of four, so the folds are `[0..3]`, `[4..7]`, `[8..11]`, `[12..15]`, `[16..19]` and *all three* positives fall in fold 5. `StratifiedKFold(5)` deals 17 negatives and 3 positives out separately, so three folds get one positive each and two get none — with only three positives and five folds it cannot do better, and scikit-learn warns that `n_splits` exceeds the members of the smallest class. Verified:

```
20 rows, 3 positives. StratifiedKFold(5) test folds:
fold 1 test [0, 1, 2, 3] positives 0
fold 2 test [4, 5, 6, 7] positives 0
fold 3 test [8, 9, 10, 17] positives 1
fold 4 test [11, 12, 13, 18] positives 1
fold 5 test [14, 15, 16, 19] positives 1
KFold(5, shuffle=False) on the same sorted array:
fold 1 test [0, 1, 2, 3] positives 0
fold 2 test [4, 5, 6, 7] positives 0
fold 3 test [8, 9, 10, 11] positives 0
fold 4 test [12, 13, 14, 15] positives 0
fold 5 test [16, 17, 18, 19] positives 3
```

What goes wrong: four of the five `KFold` folds contain no positive at all, so recall is undefined on them and accuracy is trivially high; and the fold that tests on rows 16–19 trains on a set containing no positives, so no classifier can be fitted. The deeper lesson is upstream of the splitter: with three positives, **no** cross-validation scheme will give a usable recall estimate. Collect more positives, or report the confusion matrix and refuse to report a rate.

Solution 2.

$$\begin{aligned} \text{accuracy} &= \frac{10 + 9900}{10000} = 0.9910, & \text{precision} &= \frac{10}{10 + 60} = 0.1429, \\ \text{recall} &= \frac{10}{10 + 30} = 0.2500, & F_1 &= \frac{2 \text{ TP}}{2 \text{ TP} + \text{FP} + \text{FN}} = \frac{20}{110} = 0.1818, \end{aligned}$$

and balanced accuracy = $\frac{1}{2}(9900/9960 + 10/40) = \frac{1}{2}(0.9940 + 0.2500) = 0.6220$. An always-negative model scores $9960/10000 = 0.9960$, which is *higher* than the real model’s 0.9910.

```
confusion matrix [[9900 60] [30 10]]
accuracy 0.9910 precision 0.1429 recall 0.2500 F1 0.1818
an always-negative model would score accuracy 0.9960
balanced accuracy 0.6220
```

Which number to report: none of them alone. Report the confusion matrix itself — “we catch 10 of 40 frauds and raise 60 false alarms” — and add PR-AUC if a threshold-free summary is wanted. If forced to a single rate, use recall or F_1 ; never accuracy.

Solution 3. Row-fits are $k \cdot n(1 - 1/k) = n(k - 1)$.

```
cost of k-fold on n = 5000 rows
2-fold:      2 fits, each on 2500 rows ->      5000 row-fits
```

5-fold:	5 fits, each on 4000 rows ->	20000 row-fits
10-fold:	10 fits, each on 4500 rows ->	45000 row-fits
LOO-fold:	5000 fits, each on 4999 rows ->	24995000 row-fits

At 0.4 s per fit on 4000 rows the per-row cost is 10^{-4} s, so: 2-fold 0.5 s, 5-fold 2 s, 10-fold 4.5 s, LOOCV 2499.5 s — about 42 minutes. That is the honest reason nobody uses LOOCV on five thousand rows.

Solution 4. Something like: “97.8% is the upper end of the range this experiment produces by chance — five hundred re-runs of the same code on the same data give anything from 81% to 98%, so a single split cannot distinguish your model from one that is seventeen points worse. Please report a stratified 5-fold or 10-fold mean with its standard deviation, and state the splitter and the seed.” Verified spread:

500 seeds on wine: 95% of single splits land in [0.8106, 0.9778], width 0.1672
a report of 1.0000 and a report of 0.7556 came from the same code

Solution 5. (a) `GroupKFold` or `StratifiedGroupKFold` grouped by *patient* — four images of the same chest are near-duplicates. (b) `TimeSeriesSplit` — a model that trains on next month to predict this month is not a forecaster. (c) `StratifiedKFold(5, shuffle=True)` — independent rows, and stratifying costs nothing even when the classes are near-balanced. (d) `TimeSeriesSplit` first, because fraud patterns drift and you must predict forward; then check group leakage by cardholder, and use group-aware splitting if a single cardholder carries many of the frauds. When constraints conflict, the time constraint wins — it is the one that mirrors deployment. (e) `GroupKFold` grouped by *examiner* if you are modelling marks, because examiner severity is a shared effect; group by student instead if the row unit is a question within a script.

Solution 6. $B - A = (4, 6)$, so the synthetic point is $A + 0.3(4, 6) = (2, 4) + (1.2, 1.8) = (3.2, 5.8)$.

For the one-hot block, SMOTE interpolates each coordinate independently: $(1, 0, 0) + 0.3((0, 1, 0) - (1, 0, 0)) = (0.7, 0.3, 0)$. That is a patient who is 70% blood group A and 30% blood group B — a row that cannot exist. Interpolation is only meaningful for continuous features. The remedies are SMOTE-NC (which takes the majority category among the neighbours rather than averaging), or resampling only on the numeric block, or — usually best — using class weights instead.

Solution 7. The maximum of m random variables is at least as large as any one of them, and strictly larger whenever they differ, so $\mathbb{E}[\max_i(\mu + \varepsilon_i)] \geq \mu + \mathbb{E}[\varepsilon_1] = \mu$, with strict inequality as soon as the ε_i have any spread. The selected value carries the noise that made it the winner, so re-measuring it on fresh folds regresses it back towards μ . For roughly Gaussian noise of sd σ the expected maximum grows like $\mu + \sigma\sqrt{2 \ln m}$: slowly, but without bound, so bigger grids buy bigger illusions. The measured table agrees: +0.0000 at $m = 1$, +0.0200 at 4, +0.0617 at 20, +0.0783 at 60.

Solution 8. Errors: (i) SMOTE was applied to the whole dataset before splitting, so synthetic test rows were built from training rows — the leak of Section 6.6, worth +0.7809 of F_1 in the measured case; (ii) the test folds are 50/50 after resampling, so even a leak-free score would answer the wrong question; (iii) no held-out test set exists at all, only cross-validation used both for choosing and for reporting; (iv) F_1 alone hides the precision/recall trade-off that a churn

team must actually choose. The honest number for a 2% minority is far more likely to lie in the 0.1–0.3 range than at 0.89.

Corrected procedure: (1) split off a stratified test set and put it away; (2) build a stratified k -fold loop on the training part; (3) inside each fold, fit the scaler and resample *the training part of that fold only*; (4) score on the untouched validation part, using recall, precision, F_1 and PR-AUC; (5) choose hyper-parameters — including whether to resample at all — by the inner loop, and pick the operating threshold from the precision–recall curve; (6) refit on all the training rows with the chosen settings and report on the test set, once, with the confusion matrix.

Solution 9. Two independent problems. **Time:** with `shuffle=True` on three years of hourly data, almost every test hour sits between two training hours, so the model interpolates rather than forecasts, while the deployment task is extrapolation. **Groups and features:** “hour of day” plus “day of year” plus “station id” identifies a specific hour at a specific station almost uniquely across three years, so the model can memorise a lookup table. Together these give a score that says nothing about tomorrow.

The experiment that shows it: refit with `TimeSeriesSplit(5)` and compare. Section 3.4 is exactly this experiment on synthetic data, and it moved R^2 from $+0.9975$ to -2.8167 . Then, as a second check, hold out the last three months entirely and predict them; a model that scores well on shuffled folds and badly here is memorising.

Solution 10. For logistic regression the loss is a sum of per-row terms, so multiplying a row’s term by w and including w copies of that row are algebraically the same thing, up to the randomness in which rows get copied. Hence the near-identical numbers: recall 0.7800 for both, accuracy 0.8556 against 0.8542.

For a random forest the weight enters the impurity computation at each split and the bootstrap sampling, not a smooth additive loss; and the final prediction is a majority vote at the leaves, which a weight need not change at all. That is why `class_weight="balanced"` left the forest’s predictions identical to the untreated forest (recall 0.0000 in both) while oversampling and undersampling moved them.

In production: prefer `class_weight`. It adds no rows, adds no randomness, cannot leak across a split, costs nothing in memory or time, and is a single keyword argument. Before deciding, check three things — that the model supports it (`class_weight` or `sample_weight`), that PR-AUC does not get worse (in Section 6.5 it did), and whether simply moving the threshold on the untreated model achieves the same recall more cheaply. It usually does.

10 References and further reading

All links were checked on 22 August 2026. Free resources are marked [free].

Textbooks

- Müller and Guido, *Introduction to Machine Learning with Python*, O’Reilly, 2016. **The prescribed text for this lecture.** §5.1 cross-validation — §5.1.1 cross-validation in scikit-learn, §5.1.2 the benefits of cross-validation, §5.1.3 stratified k -fold and the other splitters (leave-one-out, shuffle-split, `GroupKFold`); then §5.2 grid search, §5.2.4 nested cross-validation, and §5.3.2 on metrics for imbalanced binary classification. Notebooks [free]: https://github.com/amueller/introduction_to_ml_with_python

- James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*. [free] Ch. 5, “Resampling Methods”: §5.1.1 the validation-set approach and its variability — the same experiment as Section 2.3; §5.1.2 LOOCV; §5.1.3 k -fold; §5.1.4 the bias–variance trade-off for k ; §5.1.5 cross-validation for classification; §5.2 the bootstrap. The lab at the end of the chapter is worth doing in full. <https://www.statlearning.com/>
- Han, Kamber and Pei, *Data Mining: Concepts and Techniques*, 3rd edition, Morgan Kaufmann, 2012. §8.5 on evaluating classifier accuracy — §8.5.1 the confusion matrix and the derived measures, §8.5.2 holdout and random subsampling, §8.5.3 cross-validation, §8.5.4 the bootstrap; **§8.6** on comparing classifiers, significance tests and ROC curves; and §8.6.5 on the class-imbalance problem, which is where the oversampling/undersampling vocabulary of the syllabus comes from.
- Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, 2e, Springer. Ch. 7 is the definitive treatment: §7.10 cross-validation, and above all **§7.10.2, “The Wrong and Right Way to Do Cross-validation”**, which is Section 5 of these notes written twenty years earlier.
- Kuhn and Johnson, *Applied Predictive Modeling*, Springer. Ch. 4 on over-fitting and resampling — a very clear account of why repeated k -fold beats a single split — and Ch. 16 on remedies for class imbalance, including sampling methods and cost-sensitive training.
- Kuhn and Johnson, *Feature Engineering and Selection*. [free] §3.4 on resampling, and Ch. 10–11 on the selection bias that Lecture 6 measured and this lecture explains. <http://www.feate.engineering/>

Online courses

- **NPTEL, Introduction to Machine Learning** — Prof. Balaraman Ravindran, IIT Madras. The reference course for this semester; the evaluation-measures and cross-validation lectures use the vocabulary the examination uses. [free] <https://nptel.ac.in/courses/106106139>
- **NPTEL, Data Mining** — Prof. Pabitra Mitra, IIT Kharagpur. The classifier-evaluation lectures match Han, Kamber and Pei §8.5–8.6 section for section. [free] <https://nptel.ac.in/courses/106105174>
- **SWAYAM** — the national portal that hosts NPTEL, with certification. [free] <https://swayam.gov.in/>
- **Kaggle Learn, Intermediate Machine Learning** — seven short browser lessons; the *Cross-Validation* lesson and the *Data Leakage* lesson together are the practical companion to this entire lecture, and take about an hour. [free] <https://www.kaggle.com/learn/intermediate-machine-learning>
- **Google Machine Learning Crash Course** — the *Classification* module (thresholds and the confusion matrix; accuracy, recall, precision; ROC and AUC) and the *Datasets, generalization, and overfitting* module (imbalanced datasets; dividing the original dataset). [free] <https://developers.google.com/machine-learning/crash-course/classification>
<https://developers.google.com/machine-learning/crash-course/overfitting/imbalanced-datasets>

Videos

- StatQuest with Josh Starmer, *Machine Learning Fundamentals: Cross Validation* — six minutes, and the clearest available picture of why you rotate the test fold. Watch before Section 4. [free] <https://youtu.be/fSytzGwBVw>
- StatQuest with Josh Starmer, *Machine Learning Fundamentals: The Confusion Matrix* — the table of Section 6 built up one cell at a time. [free] <https://youtu.be/Kdsp6soqA7o>
- StatQuest with Josh Starmer, *ROC and AUC, Clearly Explained!* — what moving the threshold does, which is the operation every resampler in Section 6.3 is imitating. [free] <https://youtu.be/4jRBRDbJemM>
- Mahesh Huddar, *K-Fold Cross Validation, Stratified K-Fold, Leave-one-out Leave-P-Out Cross Validation* — the splitters worked through in examination vocabulary, with a numerical example. [free] <https://youtu.be/PF2wLKv2lsI>
- CampusX, *Imbalanced Data in Machine Learning | Undersampling | Oversampling | SMOTE* — the remedies of Section 6.3 demonstrated in Python. [free] <https://youtu.be/yh2AKoJCV3k>

Documentation and interactive explainers

- scikit-learn user guide, *Cross-validation: evaluating estimator performance* (§3.1) — the authoritative list of splitters, with the “visualizing cross-validation behavior” example that Figure 5 is modelled on; and *Tuning the hyper-parameters of an estimator* (§3.2), whose nested-CV example is Section 5. [free] https://scikit-learn.org/stable/modules/cross_validation.html
https://scikit-learn.org/stable/modules/grid_search.html
https://scikit-learn.org/stable/auto_examples/model_selection/plot_cv_indices.html
https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html
- scikit-learn, *Common pitfalls and recommended practices* — read the data-leakage section before you next report a score. [free] https://scikit-learn.org/stable/common_pitfalls.html
- scikit-learn user guide, *Metrics and scoring* (§3.4) — the definitions of precision, recall, F_β , balanced accuracy, average precision and the ROC and PR curves used throughout Section 6. [free] https://scikit-learn.org/stable/modules/model_evaluation.html
- *imbalanced-learn* documentation — the over-sampling, under-sampling and metrics chapters, and in particular §9, *Common pitfalls and recommended practices*, which demonstrates the leak of Section 6.6 on a real dataset and shows the `imblearn.pipeline.Pipeline` that prevents it. [free] https://imbalanced-learn.org/stable/common_pitfalls.html
https://imbalanced-learn.org/stable/over_sampling.html
- MLU-Explain, *Train, Test, and Validation Sets* — an interactive explainer; drag the points and watch the three sets do different jobs. The best five minutes you can spend on Section 5. [free] <https://mlu-explain.github.io/train-test-validation/>

Articles

- Chawla, Bowyer, Hall and Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique”, *Journal of Artificial Intelligence Research* **16** (2002), 321–357. [free] The paper Section 6.4 implements; the algorithm itself occupies half a page. <https://www.jair.org/index.php/jair/article/view/10302>
- Cawley and Talbot, “On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation”, *Journal of Machine Learning Research* **11** (2010), 2079–2107. [free] The definitive study of the optimism measured in Section 5.2, with the argument for nested cross-validation. <https://www.jmlr.org/papers/v11/cawley10a.html>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.; Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.; James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.; Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.